DM 3

CPES ≪ Sciences des données, arts et cultures ≫ : Introduction à l'algorithmique

Antoine GAUQUIER
antoine.gauquier@ens.fr

Pierre ABOULKER
pierreaboulker@gmail.com

18 mars 2024

1. La partie programmation du DM doit être envoyée par mail à Antoine Gauquier avant le lundi 22 avril 23h59. La partie théorique doit être rendu en main propre à Pierre Aboulker au début du cours du 23 avril.
2. Nous rappelons que le plagiat est absolument interdit : vous avez bien sûr la possibilité de discuter du sujet de DM avec vos camarades, mais tout ce que vous rédigez doit être votre propre production ; les recopies manifestes seront fortement sanctionnées.
3. Essayez d'être **rigoureux** et **bref** (dans cet ordre)
4. Le barème est donné à titre indicatif.
5. Si quelque chose n'est pas claire, n'hésitez pas à demander à Pierre si ça concerne la partie théorique, ou à Antoine pour la partie programmation. Par mail ou à la fin d'un cours.

A. Exercices théoriques, 10 points

Exercice 1 (2 points) : Montrez comment s'exécute le parcours en profondeur sur le graphe de la Figure 1. On se base sur le pseudocode du slide 22 du cours sur DFS (Algo 7 DFS et Algo 8 DFS-Visite). On suppose que la boucle *for* des lignes 5 à 7 de la procédure DFS considère les sommets dans l'ordre alphabétique, et que chaque liste d'adjacence est ordonnée alphabétiquement. Montrez les temps de découverte et de fin pour chaque sommet, et donner le type de chaque arête.

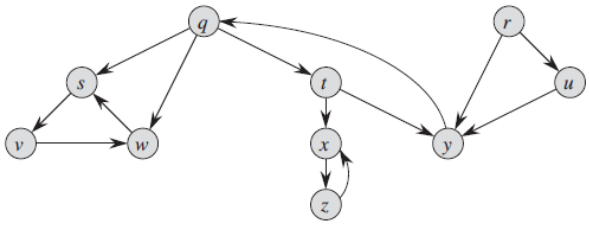


FIGURE 1 – Le graphe dirigé de l'exo 1

Exercice 2 (5 points) : Un graphe est *biparti* si ces sommets peuvent être partitionner en deux ensembles A et B tel que aucune arête n'ait ses deux extrémités dans A ou ses deux extrémités dans B (autrement dit, toutes les arêtes ont une extrémité dans A et l'autre dans B). Un graphe est 2-*colorable* si on peut "colorer" ses sommets avec deux couleurs, de façon à ce que toute pair de sommets adjacents reçoivent des couleurs différentes. Plus formellement, G est 2-colorable s'il existe une fonction $\varphi : V(G) \to \{0,1\}$ tel que pour tout arête uv, $\varphi(u) \neq \varphi(v)$.

Question 1 (0.5 point) : Soit G un graphe biparti avec bipartition (A, B). Montrer que $\sum_{v \in A} deg(v) = \sum_{v \in B} deg(v)$.

Question 2 (0.5 point) : Montrer que un graphe est biparti si et seulement si il est 2-colorable.

Question 3 (1 point) : Montrer qu'un graphe est biparti si et seulement si il n'a pas de cycle impair.

Question 4 (1.5 points) : Montrer comment utiliser DFS pour décider si un graphe est biparti, et s'il l'est renvoyer une bipartition. Attention, ici le graphe n'est pas dirigé, donc DFS est plus simple, dans le sens où il y a moins de types d'arêtes que dans le cas dirigé.

Question 5 (1.5 point) : Il existe deux types de catcheur professionnels : les "babyfaces" (les "bons") et les "heels" (les "méchants") (si vous voulez en savoir plus, aller sur la page wikipedia *Heel (catch)* :)). Entre chaque catcheur, il peut y avoir une rivalité, ou pas. Supposons qu'il y a n catcheurs et une liste de r paires de catcheurs rivales. Donnez un algorithme qui détermine s'il est possible de désigner chaque catcheur comme babyfaces ou heels de manière à ce que chaque rivalité soit entre un babyface et un heel. Donner la complexité de votre algorithme.

Exercice 3 (1 point) : Le professeur Tournesol propose la conjecture suivante. Soit $G = (V, E)$ un graphe connexe avec une fonction de poids définie sur E. Soit $A \subseteq E$ un ensemble prometteur, c'est à dire inclus dans un arbre couvrant de poids minimum de G. Soit $(S, V \setminus S)$ une coupe de G respectant A, et soit (u, v) une arête sûre pour A traversant $(S, V \setminus S)$. Alors, (u, v) est une arête minimale pour la coupe $(S, V \setminus S)$. Montrez que la conjecture du professeur est fausse en donnant un contre-exemple.

Exercice 4 (2 points) : Montrez que si une arête (u, v) est contenue dans un arbre couvrant depoids minimum, alors il existe une coupe du graphe pour laquelle elle est une arête traversante de poids minimum.

B. Exercice pratique : la couverture réseau par fibres optiques (10 points)

Le *plan France Très Haut Débit (PFTHD)*, lancé en 2013, a pour objectif d'offrir une couverture réseau par fibre optique sur l'ensemble du territoire français d'ici 2025. La couverture d'une telle zone géographique est très coûteuse, en particulier en matières premières. Un des objectifs est donc de minimiser la longueur totale de câble à déployer sous terre pour réaliser l'ensemble de la couverture. On propose alors de résoudre ce problème d'optimisation en utilisant la théorie des graphes, via la modélisation suivante.

Soit un graphe non-orienté et pondéré $G = (V, E)$, représentant la zone géographique à couvrir. Dans ce graphe, les arêtes E peuvent être vues comme des rues, des routes, ou des portions de rues ou de routes. Les sommets V représentent des *Points de Branchement Optique (PBO)*, qui sont des boîtiers sur lesquels les foyers peuvent être branchés pour être couverts par la fibre. À chaque arête $e \in E$ est associé un poids $W(e)$, avec $W : E \to \mathbb{R}_+$, donnant la longueur de câble nécessaire pour relier les extrémités de l'arête e.

L'objectif étant de relier l'ensemble des sommets de V en utilisant un minimum de longueur de câble, il suffit de calculer l'*arbre couvrant de poids minimum* (que l'on écrira par la suite ACPM) du graphe G.

Création du graphe pondéré

Afin de pouvoir résoudre le problème, il faut déjà être en mesure de représenter un graphe **non-orienté** et **pondéré**. Nous avons déjà dans les TP précédents implémenté un graphe non-orienté : il reste alors à intégrer la pondération, à savoir, être en mesure d'associer un poids à chaque arête.

Question 1 Recopier le code des classes *Sommet* et *Graphe* définies dans les TP précédents. Leur code est disponible dans la correction des TP correspondants.

Question 2 Ajouter à la classe *Graphe* une méthode appelée `ajout_arete_ponderee`, prenant en argument `id_sommet_1` et `id_sommet_2`, les `id` des deux sommets constituant l'arête, **et `poids`, le poids de l'arête**. Afin que l'ajout des arêtes soit consistant avec la classe *Sommet*, le paramètre `nouveau_voisin` de la méthode `ajout_voisin` de *Sommet* ne sera plus un entier représentant l'ID du voisin, **mais un dictionnaire contenant deux clés** : `'id'`, clé à laquelle on associera l'ID du voisin, et `'poids'`, clé à laquelle on associera le poids associée à l'arête formée par la relation de voisinage.

Afin d'implémenter cette méthode, on repartira du code de la méthode `ajout_arete`, étant donné qu'il faut juste la compléter pour intégrer la pondération.

Implémentation de l'algorithme Union–Find

Comme vu dans le cours sur les ACPM, Union–Find est un bon choix pour l'implémentation de l'algorithme de Kruskal. Nous allons donc l'implémenter, avant de finalement s'intéresser à la résolution du problème.

Question 3 Créer une nouvelle classe *Ensemble*, et y définir un constructeur (pour rappel, il s'agit de la fonction `__init__`). Vous y déclarerez les attributs *parent* et *rang*, conformément à leurs définitions dans le cours sur Union–Find (slide 17). Cette méthode prendra deux paramètres : `self` (représentant, pour rappel, la classe *Ensemble* elle-même), et `parent`, la valeur que l'on attribuera à l'attribut `parent`. La variable de class *rang* sera elle initialisée à 0.

Question 4 Écrire, **en dehors de toute classe**, une fonction `creer_ensemble`, prenant en paramètre un ID de *Sommet* `x` et retournant un objet *Ensemble* initialisé via son constructeur.

Question 5 En partant de la slide 18 du cours sur Union–Find, écrire, **en dehors de toute classe**, une méthode `trouver`, prenant pour paramètres deux variables : `ensembles`, une liste d'objets de la classe *Ensemble*, et `x`, la position de l'objet *Ensemble* sur lequel appliquer la méthode « TROUVER » de la slide 18. *Indice : Cela implique que, lorsque vous avez $x \neq x.p$ dans le cours, vous devrez écrire* `x != ensembles[x].parent` *en Python ...*

Question 6 En partant de la slide 18 du cours sur Union–Find, écrire, **en dehors de toute classe**, une méthode `lier`, prenant en paramètres : `ensembles`, une liste d'objets de la classe *Ensemble* ; et `x` et `y`, les positions des deux objets *Ensemble* dans la liste *ensembles* que vous souhaitez lier. Il vous faudra en particulier faire attention au point soulever dans l'indice de la question 4.

Question 7 En partant de la slide 18 du cours sur Union–Find, écrire, **en dehors de toute classe**, une méthode `union`, prenant en paramètres : `ensembles`, une liste d'objets de la classe *Ensemble* ; et `x` et `y`, les positions des deux objets *Ensemble* dans la liste *ensembles* dont vous souhaitez faire l'union.

Algorithme de Kruskal

Il reste donc à implémenter l'algorithme de Kruskal. Comme ce dernier a besoin de gérer des arêtes, nous allons créer une classe supplémentaire *AreteKruskal*, avant d'implémenter l'algorithme de Kruskal.

Question 8 Implémenter une classe supplémentaire *AreteKruskal*, qui contiendra trois attributs : `id_sommet_1` et `id_sommet_2`, les deux sommets que l'arête lie, et `poids`, le poids de l'arête. Vous écrirez un constructeur qui prend en paramètre `self`, `id_sommet_1`, `id_sommet_2` et `poids`, qui initialisera les attributs mentionnés précédemment.

Question 9 En partant de la slide 25 du cours sur les arbres couvrants de poids minimal, écrire, **en dehors de toute classe**, une fonction `algorithme_kruskal`, prenant en paramètre un objet *Graphe*, et retournant une liste d'objets *AreteKruskal* qui forme l'arbre de poids couvrant minimal fournit par l'agorithme de Kruskal. Afin de vous aider, un squelette vous est fourni sur le Moodle du cours, contenant entre autres la « transformation » d'un objet *Graphe* en une liste d'objets *AreteKruskal*, ainsi que le tri d'une liste d'objets que vous avez vous-même définis.

Question 10 Créer, **en dehors de toute classe**, un objet *Graphe* représentant le graphe **non-orienté et pondéré**, correspondant à une zone géographique à couvrir, présenté en Figure 2 (l'arête $(4, 6)$ est associée à un poids de

7 et l'arête $(3, 9)$ à un poids de 1). Appliquez-y ensuite l'algorithme de Kruskal implémenté, et, à partir de la liste d'objets *AreteKruskal* retournée, afficher les attributs de chaque arête de l'ACPM, ainsi que le poids total de l'arbre calculé.

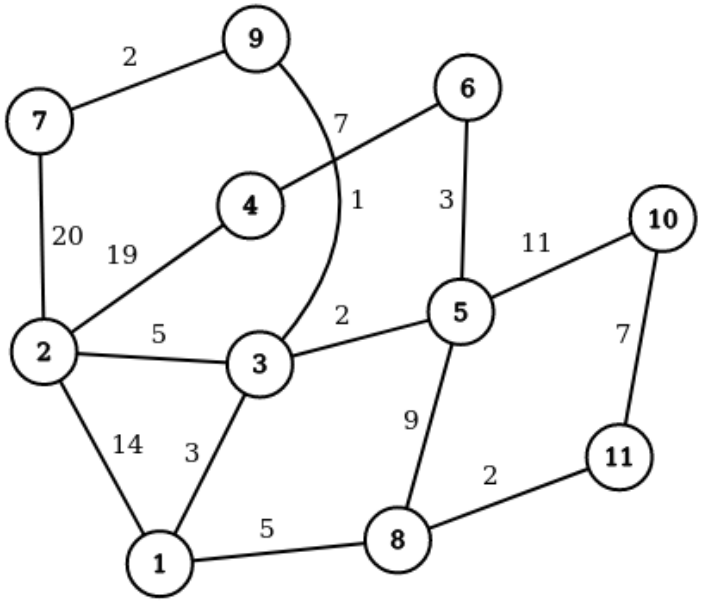


FIGURE 2 – Partie de la zone géographique à couvrir